```
    DLL(int key, int value){
        this.key = key;
        this.value = value;
        prev = null;
        next = null;
    }
}

/**
 * Your LRUCache object will be instantiated and called as such:
 * LRUCache obj = new LRUCache(capacity);
 * int param_1 = obj.get(key);
 * obj.put(key,value);
 */
```

Q. Find the next smaller element to the left for all elements of the array.

Eg : arr → [4, 5, 2, 10, 8] result → [-1, 4, -1, 2, 2]

Implemented using TreeSet().
TreeSet() stores keys in sorted order , implemented using Red-black trees
-> addition, removal, fetch take O(logN).

treeSet.lower(value) →the greatest element in this set strictly less than the given element, or null
treeSet.higher(value) → least element from the set higher than value or null.

```
public class NextGreaterLeft {
   public static void main(String[] args) {
       int[] arr = {2,5,7,3,4,9,6,3};

       int n = arr.length;
       int[] result = new int[n];
       TreeSet treeSet = new TreeSet();

       for(int i = 0 ; i < n ; i++){
           int curr = arr[i];
           result[i] = (treeSet.isEmpty() || treeSet.lower(curr) == null) ? -1 : (int) treeSet.lower(curr);
           treeSet.add(curr);
       }
       System.out.println("greatest among smaller elements in left : " + Arrays.toString(result));
   }
}
```

TC : O(N.logN)

Linked List

```
class Node{
    int val;
    Node next;
    Node(int x){
        this.val = x;
        this.next = null;
    }
}

Node start = null;
```

```
Node prev = null;
int[] arr = {1,2,4,5,6};
for(int i = 0 ; i < arr.length ; i++){
    Node newNode = new Node(arr[i]);
    if(prev != null)
        prev.next = newNode;
    else
       start = newNode;
       prev = newNode;
   }

  while(start != null) {
  System.out.println(start.val);
  start = start.next;
 }
```

Doubly linked list

```
class Node {
    int data;
    Node next;
    Node prev;

    Node(int x) {
        data = x;
        next = null;
        prev= null;
    }

}

Node constructDLL(int arr[]) {
        // Code here
        Node start = null;
        Node prev = null;
        for(int i = 0 ; i < arr.length ; i++){
            Node newNode = new Node(arr[i]);
            if(prev == null){
                start = newNode;
                prev = newNode;
            }else{
                prev.next = newNode;
                newNode.prev = prev; //only this line added in DLL
                prev = newNode;
            }
        }

        return start;
    }
```

Reverse a linked list -

```
private ListNode reverseList(ListNode node){

        if(node == null || node.next == null) //if null or single return
            return node;

        ListNode newHead = reverseList(node.next); //1<-2<-3
        node.next.next = node;
        node.next = null;

        return newHead;

    }
```

Iterative:

```
ListNode reverseList(ListNode head){
ListNode curr =  head;
ListNode prev = null;

while(curr !=null) {
      ListNode nextNode = curr.next;
      curr.next = prev;
      prev = curr;
      curr = nextNode;
}
return prev;
}
```

Merging recursive :

```
class Solution {
    public ListNode mergeTwoSortedList(ListNode list1, ListNode list2) {
        if(list1 == null)
            return list2;
        if(list2 == null)
            return list1;

        ListNode result = null;
        if(list1.val < list2.val){
            result = list1;
            result.next = mergeTwoLists(list1.next, list2);
        } else {
            result = list2;
            result.next = mergeTwoLists(list1, list2.next);
        }

        return result;
    }
}
```

Merging iterative :

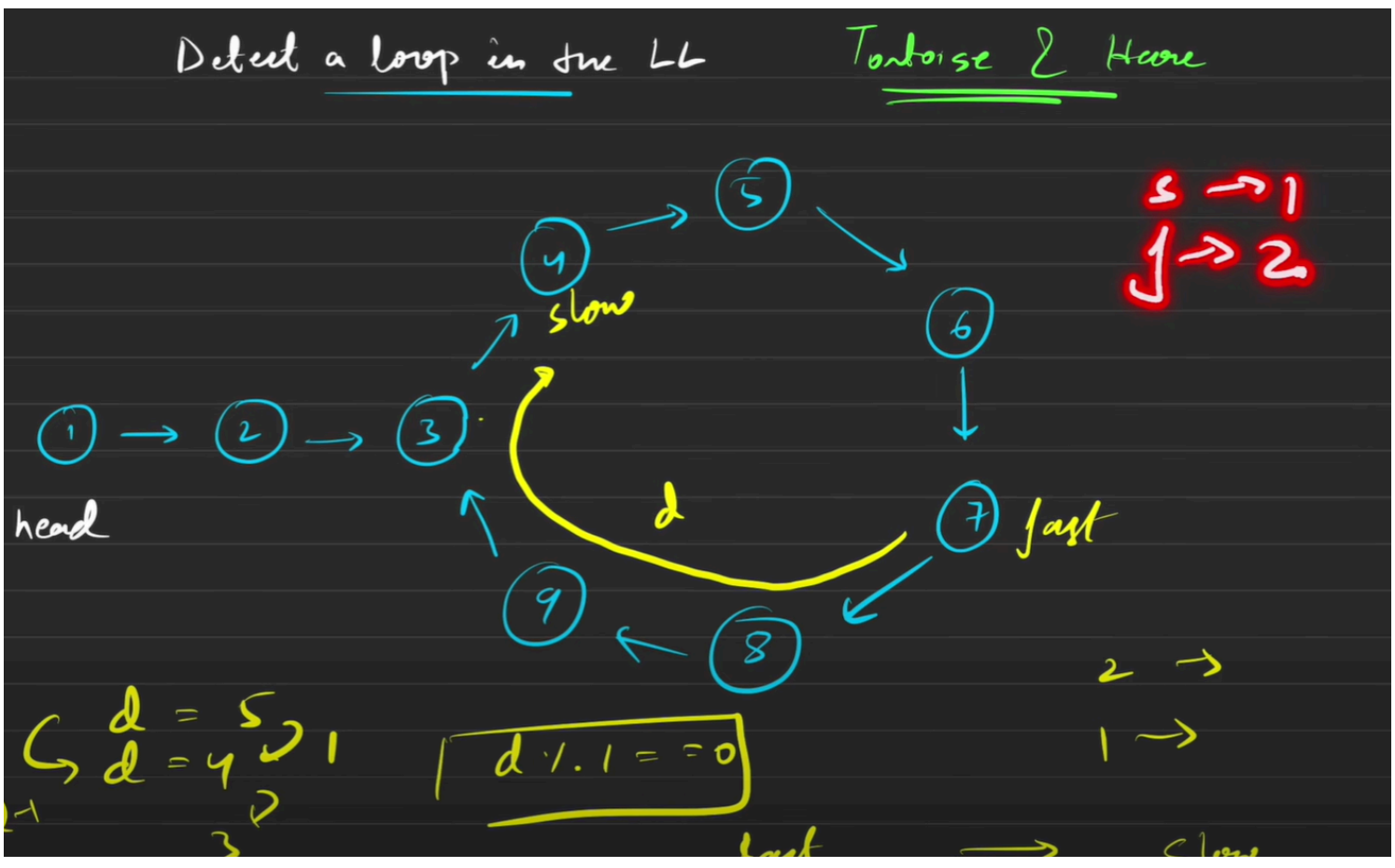
```
private Node mergeTwoSortedList(Node list1, Node list2){
        Node dummyNode = new Node(-1);
        Node res = dummyNode;
```

```
        // Merge the lists based on data values
        while (list1 != null && list2 != null) {
            if (list1.data < list2.data) {
                res.next = list1;
                res = list1;
                list1 = list1.next;
            } else {
                res.next = list2;
                res = list2;
                list2 = list2.next;
            }
            res.next = null;
        }

        // Connect the remaining
        // elements if any
        if (list1 != null) {
            res.next = list1;
        } else {
            res.next = list2;
        }

        return dummyNode.next;
    }
```

Finding Loop in linked list : IMP

We all know - Turtle and Hare algo,
Move slow = slow.next & fast = fast.next.next

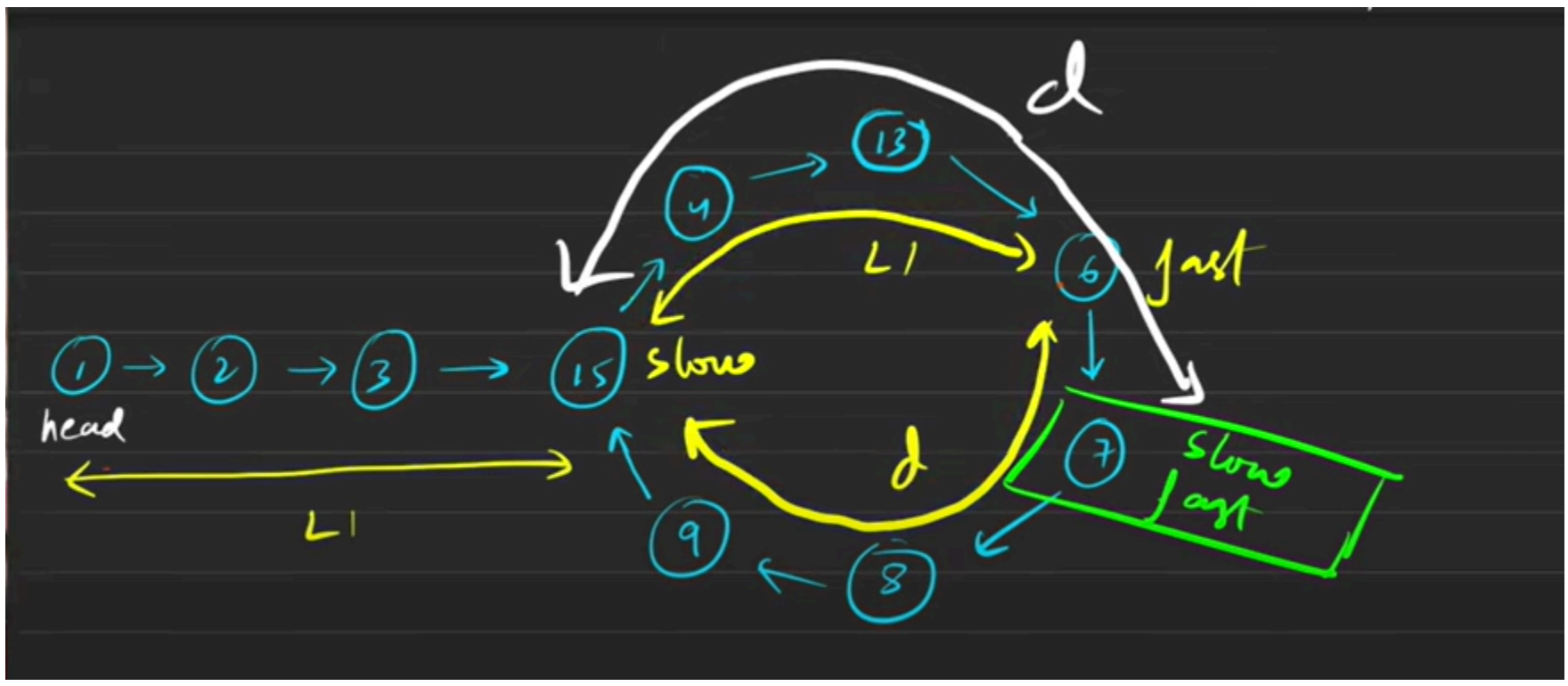
But why does it works?



Let say:
Iteration 1:
    Slow = (3) and Fast(5) , forward difference = 4 elements in b/w (6,7,8,9)
Iteration 2:
    Slow = (4) and Fast(7), forward difference = 3 elements in b/w (8,9,3)
Iteration 3:
    Slow = (5) and Fast(9), forward difference=2 elements in b/w (3,4)

Thus in every iteration distance b/w fast and slow pointers *decreases by 1.*
Thus it is bound that at some time their difference will be zero, i.e they collide.

Extension : ***finding the starting point of loop :***



d = forward distance b/w slow and fast pointer when slow is at start of loop.

- If slow covers n distance then fast will cover 2n distance,
  - *When slow at loop start* **Distance (loop_start - head) = Distance(fast - loop_start)**

- ***Slow travels a distance of 'd' when it collides with fast always,*** thus fast is left with 'L1' distance which is also the distance from source to loop start.

Interview Question: Start of Loop in a Linked List

```
public class Solution {
    public ListNode detectCycle(ListNode head) {
        if(head == null || head.next == null)
            return null;

        ListNode slow = head, fast = head;
        while(fast != null && fast.next != null){
            slow = slow.next;
            fast = fast.next.next;

            // logic to detect cycle head.
            if(slow == fast){
                slow = head;
                while(slow != fast){
                    System.out.println(slow.val + " : " + fast.val);
                    slow = slow.next;
                    fast = fast.next;
                }
                return slow;
            }

        }

        return null;
    }
}
```

Sorting Linked List:

```java
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution{
    public ListNode sortList(ListNode head) {
        // solution structure exactly like Merge sort.

        if(head == null || head.next == null)
            return head;

        ListNode midNode = findMiddle(head);

        ListNode leftStart = head;
        ListNode rightStart = midNode.next;
        midNode.next = null;       //making two list

        ListNode left = sortList(leftStart);
        ListNode right = sortList(rightStart);

        return mergeTwoSortedList(left, right);
    }

    private ListNode findMiddle(ListNode head){
        ListNode slow = head;
        ListNode fast = head.next;  //to get mid1 : else [1,3] this will again split into [1,3], [].


        while(fast != null && fast.next != null)
        {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;  //returning the Mid1
    }

    private ListNode mergeTwoSortedList(ListNode left, ListNode right){
        ListNode dummyStart = new ListNode(-1);
        ListNode temp = dummyStart;

        while(left != null && right != null){
            if(left.val < right.val){
                temp.next = left;
                temp = left;
                left = left.next;
            } else {
                temp.next = right;
                temp = right;
                right = right.next;
            }
        }
        if(left != null){
            while(left != null){
                temp.next = left;
                temp = left;
                left = left.next;
            }
        }
```

```
        if(right != null){
            while(right != null){
                temp.next = right;
                temp = right;
                right = right.next;
            }
        }

        return dummyStart.next;
    }
}
```

Q. Find the intersection of two linked list -

Sol:

Get len(List1) , len(list2)

Say len1 = 5, len2 = 3;

- Move 2 steps in list1
- then start moving one step in each and check for equal

Queue in Java

Implemented using Linked List internally in java

Deletion and Insertion takes O(1) as it maintains a start and end pointer.

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueExample {
  public static void main(String[] args) {
    // Create a queue using LinkedList
    Queue<Integer> queue = new LinkedList<>();

    // Add elements to the queue
    queue.add(10);
    queue.add(20);
    queue.add(30);
    queue.add(40);

    // Display the elements in the queue
    System.out.println("Queue: " + queue);


    // Remove elements from the queue
    int removedElement = queue.remove();
    System.out.println("Removed Element: " + removedElement);

    // Display the elements in the queue after removal
    System.out.println("Queue after removal: " + queue);

    // Peek the front element of the queue
    int frontElement = queue.peek();
    System.out.println("Front Element: " + frontElement);

    // Check if the queue is empty
    boolean isEmpty = queue.isEmpty();
    System.out.println("Is the queue empty? " + isEmpty);

    // Get the size of the queue
    int size = queue.size();
```

```
        System.out.println("Queue size: " + size);
    }
}
```

Questions :

1. **Maximum in the sliding window. LC 239 IMP**

Intuition : *use of deque (**doubly ended queue**).*

Naive approach : iteratively add elements into the window(length - k) and scan (k elements) to find the greatest.
Tc: O(N-K).(K) → if k ~ N/2. Tc ~ $O(N^2)$.

If I want to remove scan of K elements to fetch maximum value among them→ what to do?
Maintain a monotonic stack. → whenever greater/smaller in const time → monotonic stack is the go to approach.

Why use deque? → add from front and remove from back.

Arr -> [1, 3, -1, -3, 5]

Storing in deque :

Itr1 :
Deque empty → add <0, 1> → <index, value> {just for understanding, in code add only idx}

Itr2 :
Deque top (value) → 1. Current value I have '3'. No point of storing a smaller value(1) which was visited earlier in the window.

Itr3:
Deque top (value) → 3. Current value -1 , We **store** -1 because, when we move out of the window [-1,-3,5] we need the next greatest.

Hence, we maintain a decreasing monotonic stack.

Methods : Deque (Java Platform SE 8)

```java
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int len = nums.length;
        Deque<Integer> deque = new LinkedList<>();  //add and remove from both ends.
        int[] ans = new int[len - k + 1];
        int idx = 0;

        for(int i = 0 ; i < nums.length; i++) {

            int windowStartIndex = (i + 1) - k;

            //removing elements not in the sliding window.
            while(!deque.isEmpty() && windowStartIndex > deque.peekLast()) deque.removeLast();

            //remove smaller elements which can never be answer for current window
            while(!deque.isEmpty() &&  nums[i] > nums[deque.peekFirst()]) deque.removeFirst();

            deque.addFirst(i);

            if(i >= k-1) //if complete window then add the last value --> greatest among all(monotonic
decreasing queue).
                ans[idx++] = nums[deque.peekLast()];

        }
        return ans;
    }
```